

Actividad 1: Fundamentos de APIs y Estructuras JSON en Redes

1. Identificación de Pares: Enumera tres claves (keys) presentes en el nivel superior del objeto JSON.

hostname, uptime_days y managed_by_apic

2. Tipos de Datos: Identifica un ejemplo de cada uno de los siguientes tipos en el código: String, Number, Boolean y Null

String: "hostname": "R1-Core"

Number: "uptime_days": 45

Boolean: "managed_by_apic": true

Null: "description": null (en GigabitEthernet0/1)

3. ¿Qué errores de sintaxis impiden que este JSON sea válido según los estándares de red y programación?

C) Existe una coma final (trailing comma) después del valor de 'description'.

D) Faltan comas para separar los pares clave-valor después de 'vlan_id' y 'name'.

4. Reflexión Técnica: Explica por qué es crítico que un controlador como Cisco DNA Center reciba un JSON perfectamente validado antes de aplicar cambios en la red

Un controlador como Cisco DNA Center no revisa si lo que le mandas tiene sentido, solo ejecuta lo que le llega. Por eso el JSON tiene que estar bien desde el principio. Si trae una coma de más o de menos, la solicitud se rechaza o, peor, se interpreta mal y la configuración no se aplica como esperabas.

Actividad 2: Análisis de Datos de Red con Python

Parte 1: Análisis de Inventari

```
inventario = [  
    {"hostname": "Core-SW01", "ip": "10.0.0.1", "status": "up"},  
    {"hostname": "Dist-SW02", "ip": "10.0.0.2", "status": "down"},  
    {"hostname": "Access-SW03", "ip": "10.0.0.3", "status": "up"},  
    {"hostname": "Edge-R01", "ip": "172.16.1.1", "status": "down"}  
]
```

#imprmir todos los Hostname

```
print("Hostnames:")
```

for equipo in inventario:

```
    print(equipo["hostname"])
```

#imprimir todas las IPS

```
print("ip:")
```

for I in inventario:

```
    print(I["ip"])
```

#imprimir todos los que esten en down

```
print("status:")
```

for D in inventario:

```
    print(D["status"])
```

Parte 2: Comparativa de Formatos (JSON vs. XML) En redes modernas, REST APIs suelen preferir JSON por ser más ligero, pero muchos sistemas heredados o protocolos como NETCONF utilizan XML. Observa cómo se estructuran los mismos datos en ambos formatos

JSON	XML
Mas Ligero simple Tipo de datos que son números, booleanos, arreglos nativos El uso típico REST APIs Y no soporta comentarios	Mas pesado Mas estructurado y mas largo Todo es texto, se define con esquemas (XSD) NETCONF, SOAP, sistemas heredados Y si soporta comentarios